

Relatório de Android

-> Setup

Nossa, esse setup foi muito demorado e trabalhoso. Eu já tinha o Android Studio baixado em minha máquina principal, dessa forma, resolvi trabalhar com ela. Primeiramente, verifiquei como utilizar o adb: como dito nos slides, o adb já vem junto do Android Studio, porém, para utilizá-lo, eu tinha que acessar o diretório onde ele estava. Assim, resolvi adicionar o adb ao PATH, para diminuir o trabalho em relação ao seu acesso.

Com o adb configurado e funcionando, me voltei a entender o funcionamento dos emuladores android e o “avd”. O comando “emulador”, utilizado para criar um emulador, também só funcionava no diretório em que estava seu aplicativo, assim, também o coloquei no PATH de meu computador. Criando alguns emuladores de teste, percebi que emuladores criados com a Play Store não permitiam o comando “adb root”, afirmando que é de “production builds”, dessa maneira, passei a criar devices que não continham a Play Store mas possuíam a Google API, como requisitado pelos slides de Setup. A partir disso, consegui abrir os simuladores e rodar o comando adb sobre eles.

Em seguida, foquei em criar o certificado no BurpSuite para associação com o emulador. O processo de criação e modificação do arquivo DER e PEM do certificado para sua versão nomeada com hash foi tranquilo e ocorreu sem problemas, e a associação desse arquivo com o emulador teve diversos problemas, mas foi resolvido com alguns reboots e tentativas repetidas.

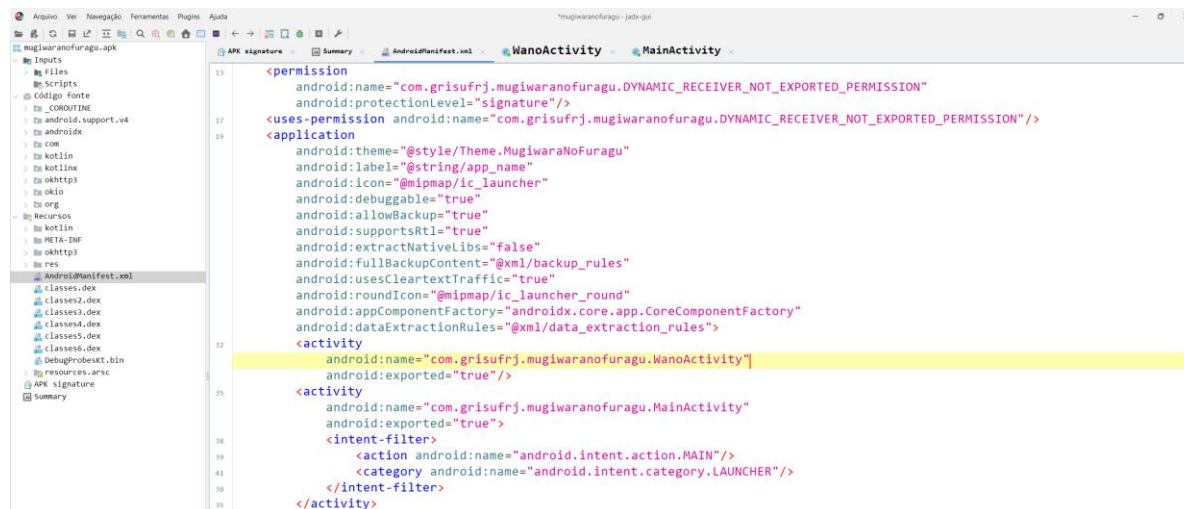
- Sunny Storage Room

Segundo a descrição do chall no site do GRIS, a flag estava guardada no armazenamento do apk fornecido. Dessa forma, baixei o apk no emulador e utilizei o adb como root para acessar o shell do emulador, conseguindo adentrar nas pastas de data e aplicativos instalados. Entrando na parte relacionada ao apk “mugiwaranofuragu”, analisei cada uma de suas pastas e, dentro da pasta “shared_prefs”, no arquivo “ThousandSunny.xml”, encontrei a flag. Imagem do terminal abaixo:

```
PS C:\Users\guilh\Downloads> adb shell
emu64xa:/ # cd /data/data/com.grisufrrj.mugiwaranofuragu
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu # ls
cache code_cache files shared_prefs
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu # cd cache
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/cache # ls
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/cache # cd ..
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu # cd code_cache
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/code_cache # ls
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/code_cache # cd ..
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu # cd files
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/files # ls
profileInstalled
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/files # cat profileInstalled
ϕLYKϕemu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/files # cd ..
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu # cd shared_prefs
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/shared_prefs # ls
ThousandSunny.xml
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/shared_prefs # cat ThousandSunny.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="flag1">CTF-BR{m4yb3-sunny-1snt-s3cur3}</string>
</map>
emu64xa:/data/data/com.grisufrrj.mugiwaranofuragu/shared_prefs # |
```

- Gear Fifth

Segundo a descrição desse chall, a flag também pode encontrada no aplicativo “mugiwaranofuragu” fornecido no desafio anterior. Para realização de uma análise diferente da feita no chall anterior, dessa vez baixei o Jadx e utilizei-o para analisar o código fonte do apk.



Dentro da seção “AndroidManifest.xml”, foi possível observar os activities que ocorrem dentro do aplicativo. Adentrando na activity “MainActivity”, verificou-se que se tratava da criação da flag 1, que constituía o desafio feito anteriormente. Por outro lado, analisei também a atividade “WanoActivity”, que apresenta o nome “Wano” citado na descrição do chall atual. Dentro desse código, vi uma função relacionada à flag 2, que era fornecida em um joguinho dentro do aplicativo. Olhando essa função, verifiquei que ela dependia de uma requisição https, imagem abaixo, assim parti para utilizar o BurpSuite e o emulador em sua versão com proxy ativado.

```
/* Loaded from: classes4.dex */
14 public class ApiClient {
15     public static boolean getFlag2(Context context) {
16         String SERVER_URL = SecureHandle.decrypt(context.getString(R.string.value2), context.getString(R.string.key2));
17         final OkHttpClient client = new OkHttpClient();
21         final Request request = new Request.Builder().url(SERVER_URL).build();
23         final AtomicBoolean win = new AtomicBoolean(false);
24         final CountDownLatch latch = new CountDownLatch(1);
38         new Thread(new Runnable() { // from class: com.grisufjrj.mugiwaranofuragu.utils.ApiClient$$ExternalSyntheticLambda0
39             @Override // java.lang.Runnable
40             public final void run() {
41                 ApiClient.lambda$getFlag2$0(OkHttpClient.this, request, win, latch);
42             }
43         }).start();
44         try {
45             latch.await();
46         } catch (InterruptedException e) {
47             e.printStackTrace();
48         }
49         return win.get();
50     }
51 }

27 static /* synthetic */ void lambda$getFlag2$0(OkHttpClient client, Request request, AtomicBoolean win, CountDownLatch latch) {
28     try {
29         try {
30             Response response = client.newCall(request).execute();
31             if (response.isSuccessful()) {
32                 response.body().string();
33                 win.set(true);
34             } catch (IOException e) {
35                 Log.e("API Error", e.getMessage());
36             } finally {
37                 latch.countDown();
38             }
39         }
40     }
41 }
```

Ai eu desisti, porque não encontrei uma forma de acessar o WanoActivity do próprio aplicativo, sem contar que o Burp se comportava estranho e não parecia conseguir ajudar em relação às requisições